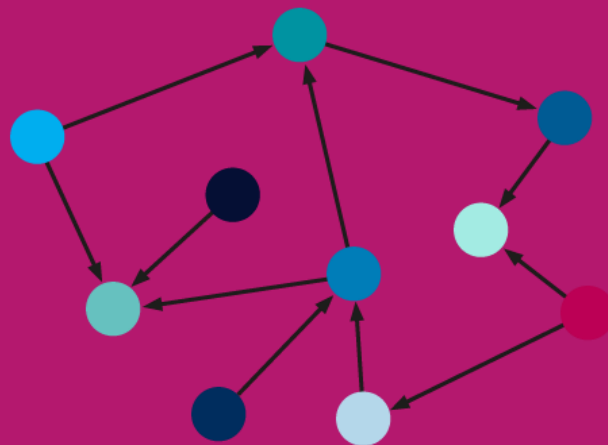
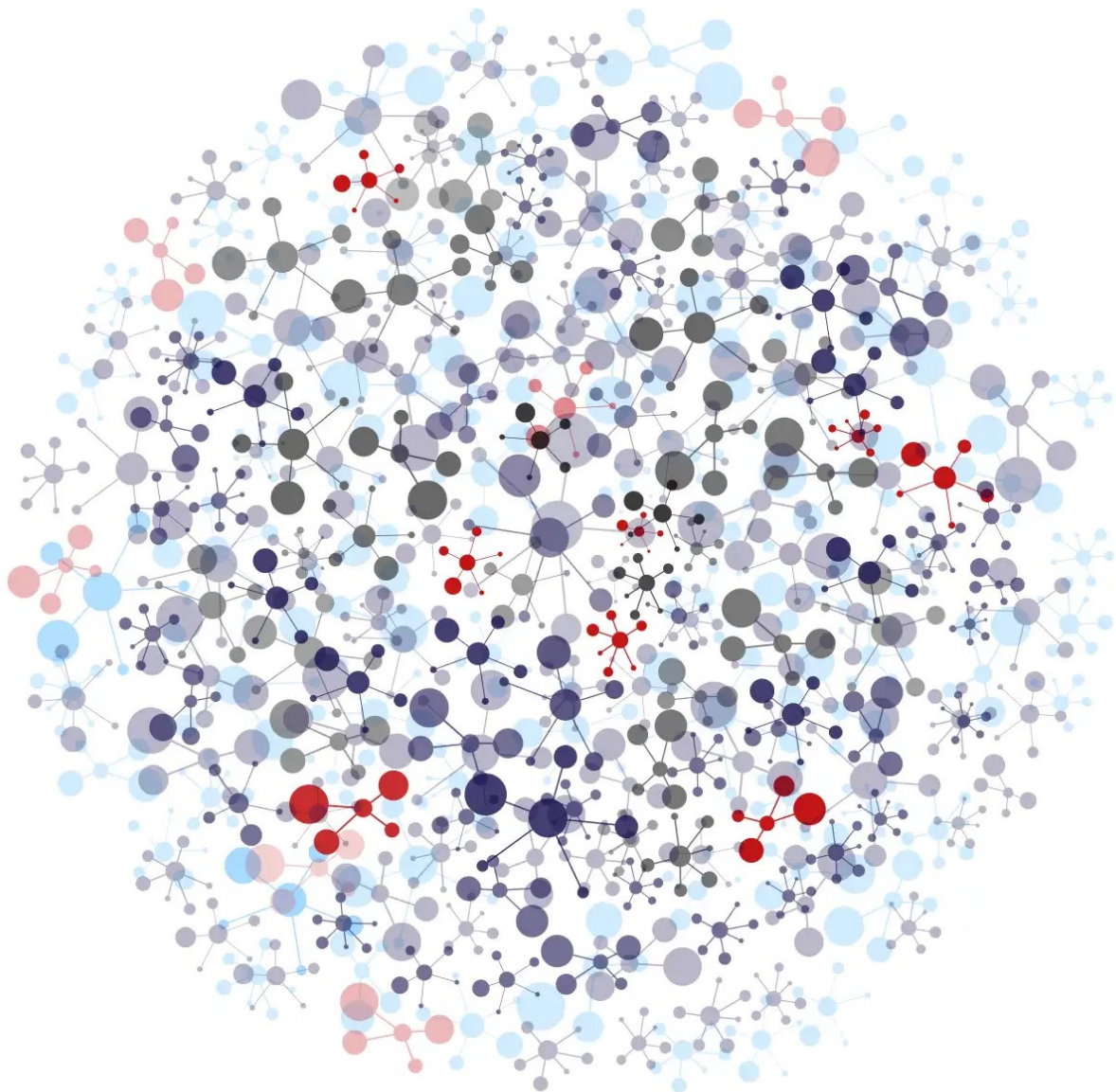




Neo4j & Cypher

Жишкович Андрій Володимирович

Графові бази даних



Графові бази даних — це тип баз даних, які використовують графову структуру для зберігання, управління та запиту даних.

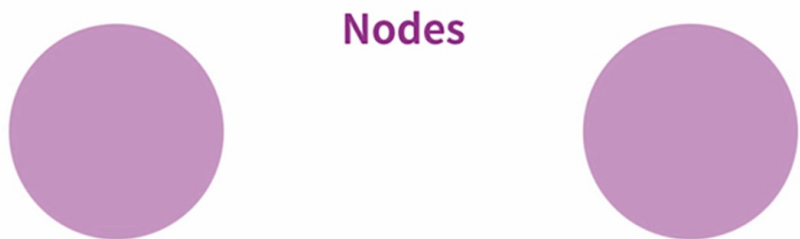
Якщо ви стикаєтеся з обмеженнями традиційних баз даних, особливо коли мова йде про складні взаємозв'язки даних, графові бази даних можуть стати саме тим рішенням, яке вам потрібно.

Вони є потужним інструментом для зберігання та обробки даних, які мають складні взаємозв'язки.

Графові бази даних часто мають спеціалізовану мову запитів для полегшення обходу зв'язків.

Анатомія властивостей графа – ключові компоненти

Перш за все, у нас є окремі **сутності**, які ми називаємо **вузлами**.



Потім у нас є з'єднання, яке об'єднує вузли, його ми називаємо **відношенням**, або **ребром**.



Відношення завжди матимуть **напрямок**, коли ми їх додаємо, але нам не обов'язково потрібно використовувати напрямок, коли ми запитуємо наші дані.

Потім у нас є спосіб категоризації вузлів, які називаються **мітками**, і спосіб категоризації відношень, який називається **типом**.



Ми можемо додатково описати наш граф за допомогою **властивостей**. Ми використовуємо **конструкцію пари ключ-значення** для їх опису, і властивості можуть бути застосовані як до вузлів, так і до відношень.



Анатомія властивостей графа – приклад

Сценарій: Джейн, клієнтка магазину, купує сонцезахисні окуляри 5 січня 2022 року.

Jane, a customer, buys sunglasses, a type of product, on January 5, 2022.



buys



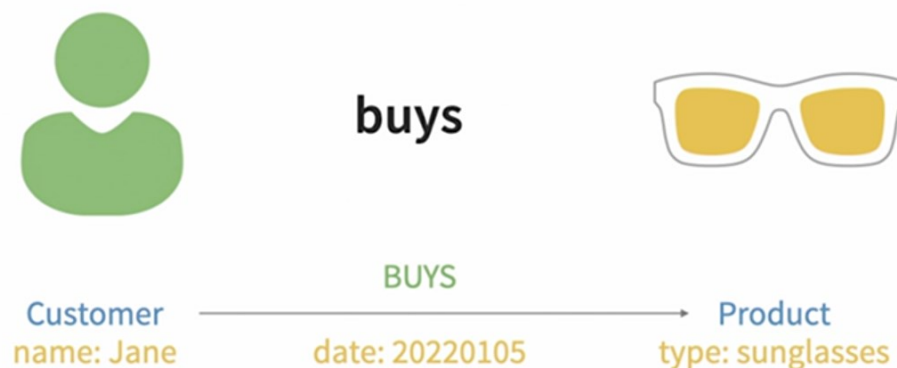
Анатомія властивостей графа – приклад

Давайте розділимо цей сценарій на компоненти:

У нас є **клієнти**, **категорія**, **продукти що купують**, якась **інша категорія**.

Джейн є клієнткою, сонцезахисні окуляри є продуктом, і зазвичай клієнти купують продукти.

Jane, a **customer**, **buys sunglasses**, a **product**, on **January 5, 2022**.



У цьому конкретному прикладі ми знаємо, що є **клієнтка на ім'я Джейн**, є **продукт типу сонцезахисні окуляри**, і що ця конкретна **покупка відбулася 5 січня 2022**.

І не менш важливе, **Джейн купує сонцезахисні окуляри**, а не сонцезахисні окуляри купують Джейн. Тож у нас є стрілка, яка вказує, хто володіє цим відношенням між клієнтом і продуктом.

Анатомія властивостей графа – приклад

Тепер давайте замінимо наші зображення на щось, що виглядає трохи більше як граф.

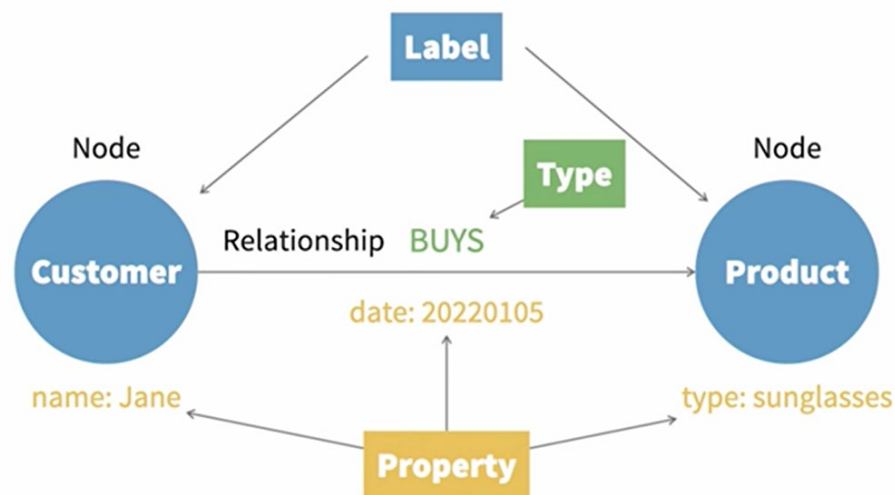
Ми замінили Джейн і сонцезахисні окуляри на **вузли**.

Дію купівлі позначили стрілкою, що є **відношенням**.

Ми також додали додаткову інформацію до нашого графа за допомогою **властивостей**.

І ми також додали **мітки** до наших вузлів: клієнт і продукт.

Jane, a **Customer**, **BUYS** sunglasses, a **Product**, on **January 5, 2022**.



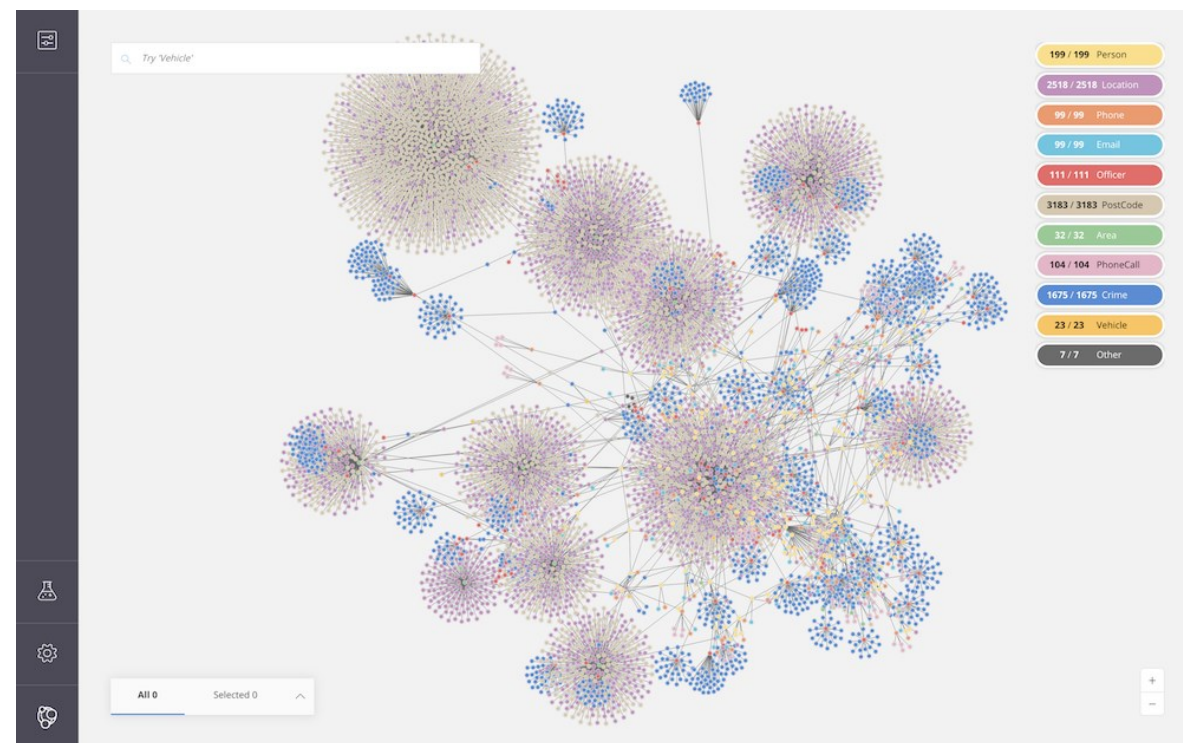
Neo4j



Neo4j — це провідна графова база даних, яка використовує унікальну модель даних для зберігання та запитів складних взаємозв'язків між точками даних. Реалізована на Java
Початковий випуск – 2007 рік.
Версія 1.0 була випущена в лютому 2010 року.

Вона оптимізована для роботи з даними, які мають складні взаємозв'язки.

Завдяки своїй здатності керувати великими обсягами взаємопов'язаних даних, Neo4j стала популярним вибором для різних галузей, від соціальних мереж і електронної комерції до охорони здоров'я та фінансів.



Переваги Neo4j

Продуктивність: Висока продуктивність під час роботи з глибоко пов'язаними даними. Продуктивність залишається стабільною навіть у випадку великих обсягів даних.

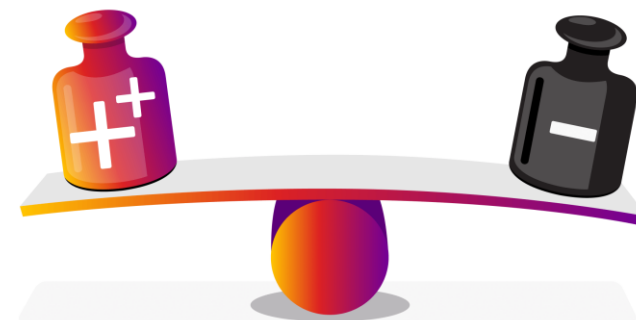
Гнучкість: Природна та інтуїтивно зрозуміла графова модель даних. Легко моделювати та змінювати структуру даних.

Потужна модель запитів: Інтуїтивна графова модель запитів, підходить для об'єктно-орієнтованих, напівструктурованих та мережевих даних. Дозволяє писати складні високопродуктивні запити.

Легка у вивченні мова запитів: Cypher — декларативна мова запитів, ефективна та легка для читання.

ACID-сумісність: Підтримка властивостей ACID (атомарність, узгодженість, ізоляція, довговічність) та повна підтримка транзакцій.

Активна спільнота: Велика та активна спільнота користувачів та розробників, що забезпечує підтримку та розвиток.



Недоліки Neo4j

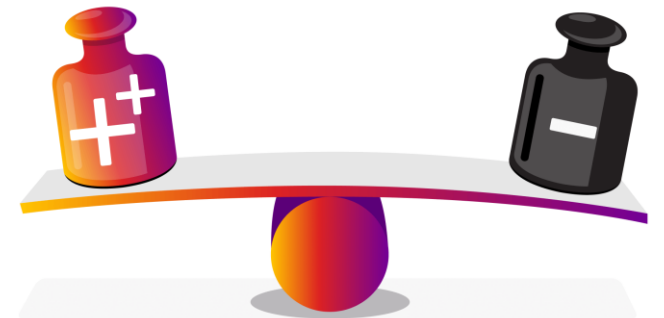
Обмежена масштабованість записів: Neo4j підтримує кластери з високою доступністю (HA) з майстер-слейв архітектурою, що добре масштабує читання, але не дуже добре масштабує запис. Високі навантаження на запис можуть вимагати вертикального масштабування майстер-інстансу.

Обмеження на зберігання: Існує верхня межа для розміру графа, що може бути обмеженням для дуже великих графів.

Крива навчання: Складність освоєння для новачків.

Вартість: Висока вартість комерційної версії Neo4j Enterprise Edition.

Інтеграція: Може вимагати додаткових зусиль для інтеграції з іншими системами.



Початок роботи з Neo4j

Завантажте відповідний інсталятор з neo4j.com/download, встановіть і запустіть **Neo4j Desktop** для створення та управління базами даних.

Neo4j підтримує Windows, macOS та Linux.

Інструменти та інтерфейси

Neo4j Browser: Веб-інтерфейс для виконання запитів Cypher та візуалізації графів.

Neo4j Desktop: Повнофункціональний клієнт для управління базами даних Neo4j.

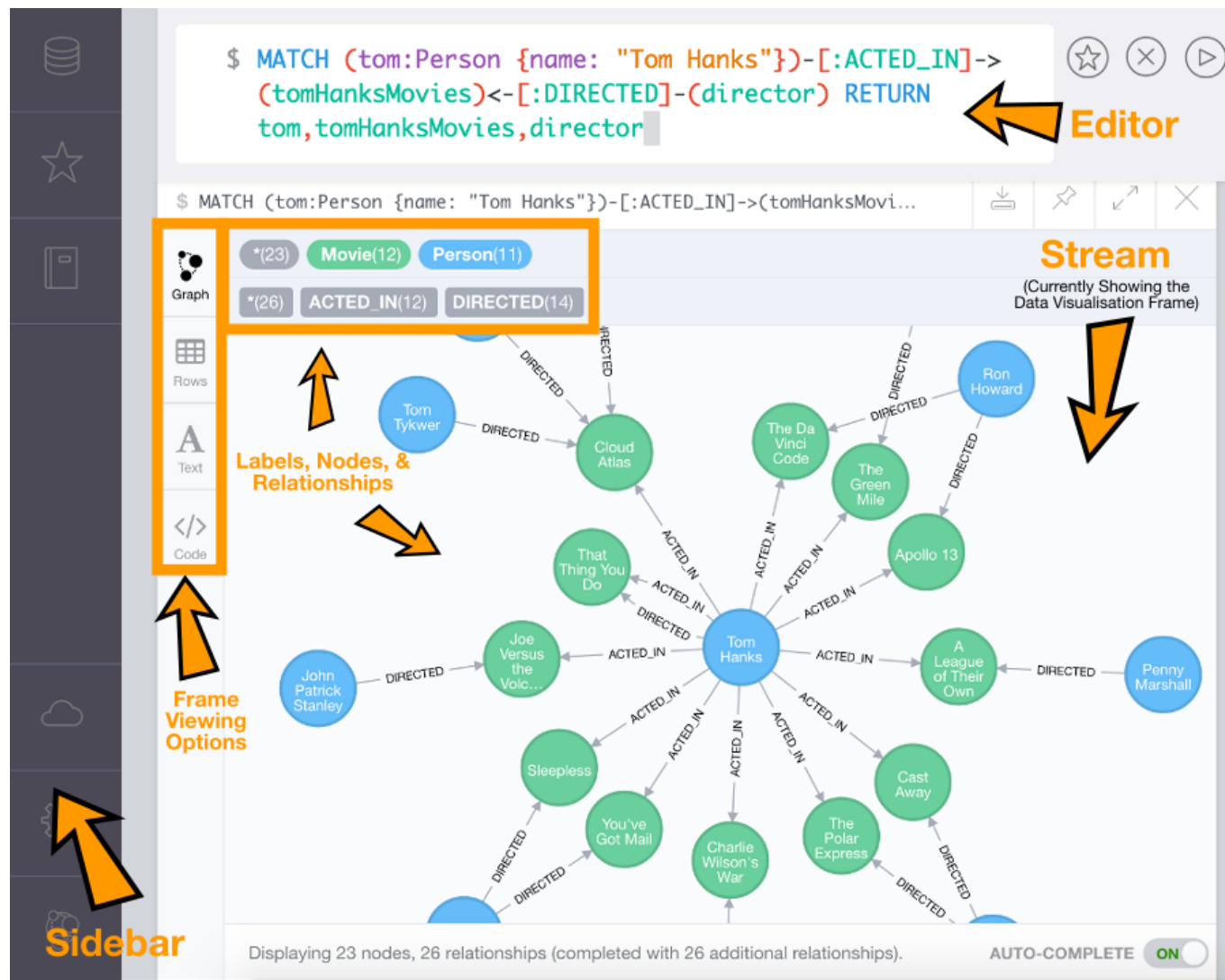
Cypher Shell: Командний інтерфейс для виконання запитів Cypher.

Neo4j Bloom: Інструмент для візуалізації та дослідження графових даних.

Neo4j Aura: Керована хмарна служба Neo4j.

Neo4j Browser інтерфейс

Neo4j Browser можна використовувати для додавання даних, виконання запитів, створення зв'язків тощо. Він також забезпечує простий спосіб візуалізації даних у базі даних.



Editor (Редактор)

Введення запитів та команд.

Stream (Потік)

Відображення результатів запитів.

Labels, Nodes, & Relationships (Мітки, Вузли та Відношення)

Представлення даних у базі даних.

Sidebar (Бічна панель)

Доступ до різних опцій, таких як деталі бази даних, налаштування браузера, документація.

Frame Viewing Options (Опції перегляду фреймів)

Перегляд даних у різних форматах.

Значення та типи

Cypher підтримує різноманітні типи даних.

Під час написання запитів Cypher неможливо оголосити тип даних.

Натомість Cypher автоматично визначає тип даних для заданого значення.

Типи даних у Neo4j

Прості типи даних: Цілі числа, числа з плаваючою комою, рядки, булеві значення.

Темпоральні значення: Дата, час, дата і час, тривалість.

Просторові значення: Географічні та картезіанські точки.

Структурні значення: Вузли, відношення, шляхи.

Списки та карти: Впорядковані колекції значень та колекції пар ключ-значення.

Null: Відсутність значення або невідоме значення.

Значення та типи

Категорія	Тип даних	Опис	Приклад
Прості типи даних	Цілі числа (Integer)	Цілі числа без десяткових знаків	42, -7
	Числа з плаваючою комою (Float)	Числа з десятковими знаками	3.14, -0.001
	Рядки (String)	Текстові дані	'Hello, world!', 'Neo4j'
	Булеві значення (Boolean)	Логічні значення true або false	true, false
Тимчасові типи	Дата (Date)	Представляє дату	date('2023-10-01')
	Час (Time)	Представляє час	time('12:34:56')
	Дата і час (DateTime)	Представляє дату і час	datetime('2023-10-01T12:34:56')
	Місцевий час (LocalTime)	Представляє час без часової зони	localtime('12:34:56')
	Місцевий дата і час (LocalDateTime)	Представляє дату і час без часової зони	localdatetime('2023-10-01T12:34:56')
	Тривалість (Duration)	Представляє тривалість часу	duration('P5DT4H') (5 днів і 4 години)
Просторові значення	Точка (Point)	Представляє географічну або картезіанську точку	point([latitude: 37.7749, longitude: -122.4194]) (географічна точка) point({x: 2.0, y: 3.0}) (картезіанська точка)
Структурні значення	Вузли (Node)	Представляють сутності в графі	
	Відношення (Relationship)	Представляють зв'язки між вузлами	
	Шляхи (Path)	Представляють послідовність вузлів і відношень	
Списки	Списки (Lists)	Представляють впорядковані колекції значень	[1, 2, 3], ['Alice', 'Bob', 'Charlie']
Кarti	Кarti (Maps)	Представляють колекції пар ключ-значення	{name: 'Alice', age: 30}
Null	Null	Представляє відсутність значення або невідоме значення	
Перетворення значень	Перетворення значень даних (Casting data values)	Перетворення значень з одного типу в інший	toInteger('42'), toFloat('3.14')
Вирази типу предикатів	Вирази типу предикатів (Type predicate expressions)	Використовуються для перевірки типу значення	exists(n.name), n.age IS NULL

Випадки використання Neo4j

Випадок використання	Опис	Галузі застосування
Соціальні мережі	Представлення та запити складних взаємозв'язків між користувачами, таких як друзі, підписники та спільні зв'язки. Використовується для персоналізованих рекомендацій, соціальних графів та стрічок активності.	Соціальні мережі, електронна комерція
Виявлення шахрайства	Аналіз взаємозв'язків між транзакціями, користувачами та пристроями для виявлення шаблонів та аномалій.	Банківська справа, страхування, електронна комерція
Рекомендаційні системи	Надання персоналізованих рекомендацій на основі поведінки користувачів та взаємодій з елементами.	Електронна комерція, розваги, соціальні мережі
Мережеві та ІТ-операції	Моніторинг та управління складною інфраструктурою та системами. Візуалізація та аналіз взаємозв'язків, виявлення вузьких місць та оптимізація продуктивності.	Телекомунікації, хмарні обчислення, корпоративні ІТ
Управління знаннями	Захоплення, організація та обмін знаннями в організації. Легке навігація та знаходження інформації.	Корпоративні знання, дослідження та розробки
Логістика та управління ланцюгами постачання	Управління рухом товарів та матеріалів через складну мережу постачальників, дистриб'юторів та роздрібних торговців. Візуалізація та оптимізація потоку товарів.	Виробництво, роздрібна торгівля, транспорт
Управління ідентифікацією та доступом	Управління доступом до ресурсів та даних в організації. Ефективні, масштабовані та безпечні системи IAM (Identity and Access Management).	Корпоративні ІТ, безпека
Охорона здоров'я	Управління та аналіз взаємозв'язків між пацієнтами, медичними працівниками, медичними записами та лікуваннями.	Охорона здоров'я, медичні дослідження
Обробка природної мови	Аналіз та моделювання взаємозв'язків між словами, концепціями та ідеями у текстових даних. Отримання інсайтів, узагальнення контенту та генерація рекомендацій.	Маркетинг, медіа, обслуговування клієнтів

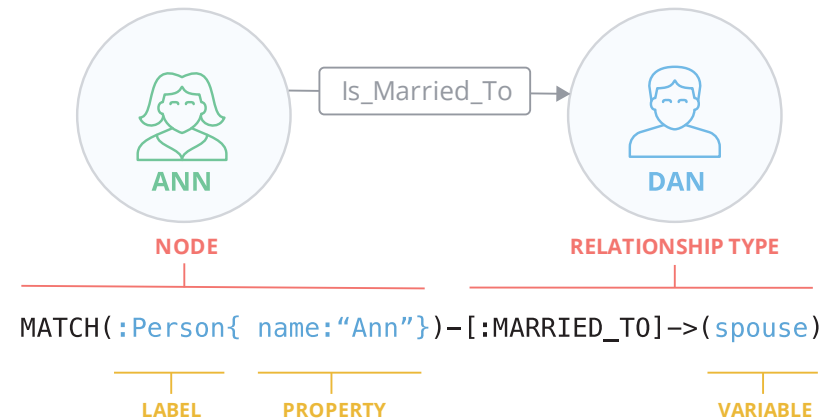
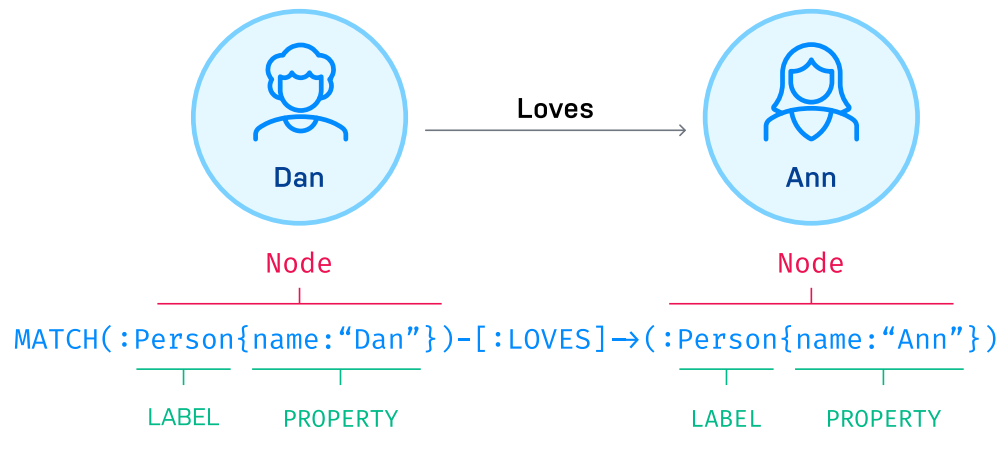
Мова запитів Cypher

Мова запитів Cypher

Cypher — це декларативна мова запитів, розроблена спеціально для роботи з графовими базами даних, такими як Neo4j.

Вона дозволяє легко створювати, читати, оновлювати та видаляти дані в графі.

Cypher нагадує SQL, але оптимізована для роботи з графовими структурами.



Cypher стає дедалі популярнішою мовою запитів для графових баз даних завдяки своїй простоті та потужності. Її підтримка розширилася на інші платформи, що робить її універсальним інструментом для роботи з графовими даними.

OpenCypher

OpenCypher — це проект з відкритим вихідним кодом, який має на меті зробити мову запитів Cypher стандартом для роботи з графовими базами даних.

Проект був ініційований компанією Neo4j, яка спочатку розробила Cypher, і тепер підтримується спільнотою розробників та компаній (Neo4j, SAP HANA Graph, Spark, RedisGraph, AgensGraph, Memgraph та інші).

Основні цілі:

Стандартизація: Встановлення єдиного стандарту для мови запитів до графових баз даних.

Відкритість: Надання відкритого доступу до специфікацій мови Cypher.

Розширення екосистеми: Підтримка інтеграції Cypher з іншими графовими базами даних та інструментами.

Основні компоненти:

Специфікація мови: Опис синтаксису і семантики Cypher.

Тестовий набір: Тести для перевірки сумісності реалізацій Cypher.

Бібліотеки та інструменти: Програмні бібліотеки та інструменти для роботи з Cypher.

Шаблони в Cypher

У Cypher все є шаблоном.

Це означає, що будь-яка структура даних, з якою ви працюєте, представлена у вигляді шаблону.

Шаблони дозволяють легко описувати та маніпулювати графовими даними.

Ось основні типи шаблонів у Cypher:

- Вузол (Node)
- Вузол з міткою (Labeled Node)
- Вузол з властивостями (Node with Properties)
- Відношення (Relationship)
- Шлях (Path)

Шаблони в Cypher – Вузли в Cypher

Як ми представляємо вузли в Cypher:

Вузли: Представляються круглими дужками `()`.

Посилання на вузол: Цей ідентифікатор вузла використовується для взаємодії з вузлом під час запитів. Не є частиною збережених даних.

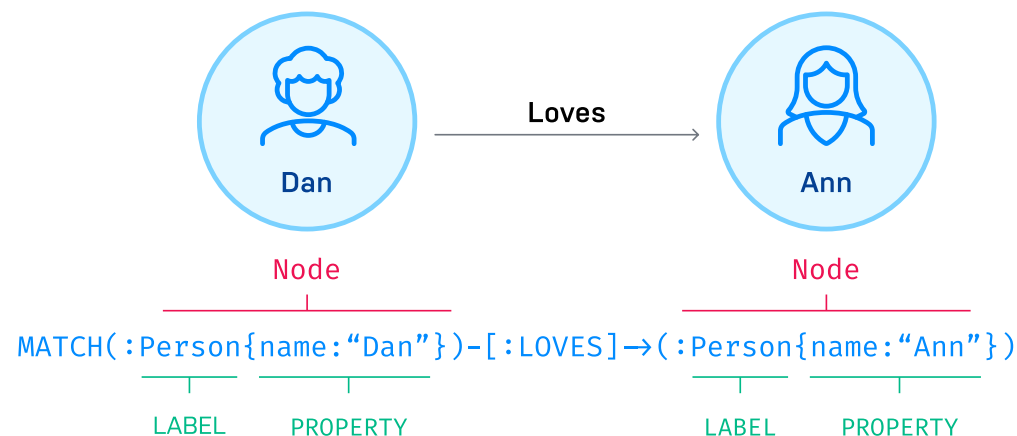
Наприклад, `(referenceA)`

Мітки вузлів: Використовуються для категоризації вузлів (групування подібних). Вказуються після двокрапки `:`.

Наприклад, `(referenceA:Label)`

Властивості вузлів: Вказуються в фігурних дужках `{}` у вигляді пар ключ-значення.

Наприклад, `(referenceA:Label {propertyKey: value})`



Шаблони в Cypher – Відношення в Cypher

Як ми представляємо відношення в Cypher:

Відношення: Представляються квадратними дужками `[]`.

Наприклад, `[r]`

Напрямок відношення: Вказується стрілками `-->` або `<--`.

Наприклад, `(a)-[r]->(b)`

Тип відношення: Вказується після двокрапки `:` всередині квадратних дужок.

Наприклад, `[<:TYPE>]`

`(a)-[:FRIEND]->(b)`

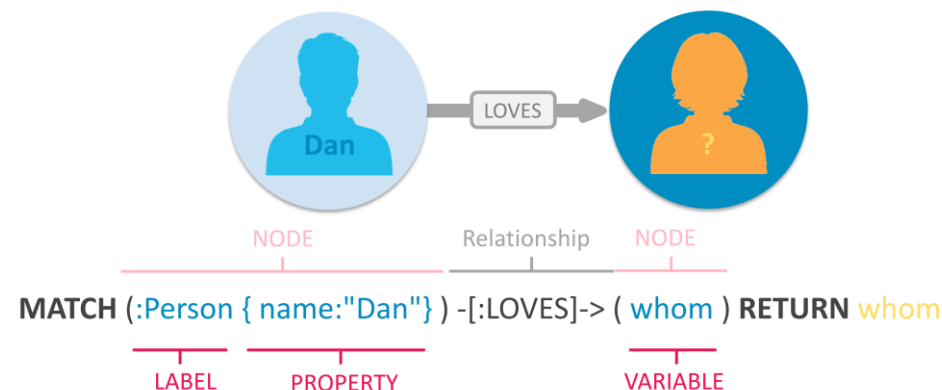
Відношення без напрямку та типу: Якщо вас цікавить лише з'єднання вузлів без напрямку та типу, використовуйте два тире `--`.

Наприклад, `(a)--(b)`

Властивості відношення: Вказуються в фігурних дужках `{}` у вигляді пар ключ-значення.

Наприклад, `[referenceB:TYPE {propertyKey: value}]`

`(a)-[r:FRIEND {since: 2020}]->(b)`



Шаблони в Cypher – завдання

Завдання:

Створити деякі дані на основі наступного твердження:

Ви, учень з ім'ям (ваше ім'я), кажете привітання з повідомленням "Радий зустрічі!"

Після того, як ви це зробите, треба виконати запит до цих даних, щоб знайти всіх учнів.

Виконання:

```
CREATE (learner:Learner {name: 'Андрій'})-[:SAYS]->(greeting:Greeting {message: 'Радий зустрічі!'})
```

```
MATCH (learner:Learner)
```

```
RETURN learner
```

Пояснення:

Створення даних:

CREATE — команда для створення нових вузлів та відношень.

`(learner:Learner {name: 'Андрій'})` — створює вузол з міткою **Learner** та властивістю **name**, яка містить ім'я "Андрій".

`[:SAYS]` — створює відношення типу **SAYS**.

`(greeting:Greeting {message: 'Радий зустрічі!'})` — створює вузол з міткою **Greeting** та властивістю **message**, яка містить повідомлення "Радий зустрічі!".

Запит для знаходження всіх учнів:

MATCH `(learner:Learner)` — знаходить всі вузли з міткою **Learner**.

RETURN `learner` — повертає знайдені вузли.

Основні команди Cypher

CREATE: Створення вузлів і відношень.

MATCH: Пошук вузлів і відношень.

WHERE: Додає обмеження до шаблону.

RETURN: Повернення результатів запиту.

SET: Оновлення властивостей вузлів і відношень.

ORDER BY: Сортування результатів запиту.

MERGE: Знаходження існуючих або створення нових вузлів і шаблонів.

DELETE: Видалення вузлів і відношень.

DETACH DELETE: Видалення вузлів разом з усіма їхніми відношеннями.

REMOVE: Видалення властивостей і міток з вузлів.

Основні команди Cypher з прикладами

CREATE: Створення вузлів і відношень.

```
CREATE (n:Person {name: 'Alice', age: 30})
CREATE (n)-[:FRIEND]->(m:Person {name: 'Bob'})
```

MATCH: Пошук вузлів і відношень.

```
MATCH (n:Person {name: 'Alice'})
RETURN n
```

RETURN: Повернення результатів запиту.

```
MATCH (n:Person)
RETURN n.name, n.age
```

WHERE: Додає обмеження, критеріїв

```
MATCH (n:Person)
WHERE n.age > 30
RETURN n.name, n.age
```

SET: Оновлення властивостей вузлів і відношень.

```
MATCH (n:Person {name: 'Alice'})
SET n.age = 31
```

ORDER BY: Сортування результатів запиту.

```
MATCH (n:Person)
RETURN n.name, n.age
ORDER BY n.name
```

MERGE: Знаходження існуючих або створення нових вузлів і шаблонів.

```
MERGE (n:Person {name: 'Alice'})
```

DELETE: Видалення вузлів і відношень.

```
MATCH (n:Person {name: 'Alice'})
DELETE n
```

DETACH DELETE: Видалення вузлів разом з усіма їхніми відношеннями.

```
MATCH (n:Person {name: 'Alice'})
DETACH DELETE n
```

REMOVE: Видалення властивостей і міток з вузлів.

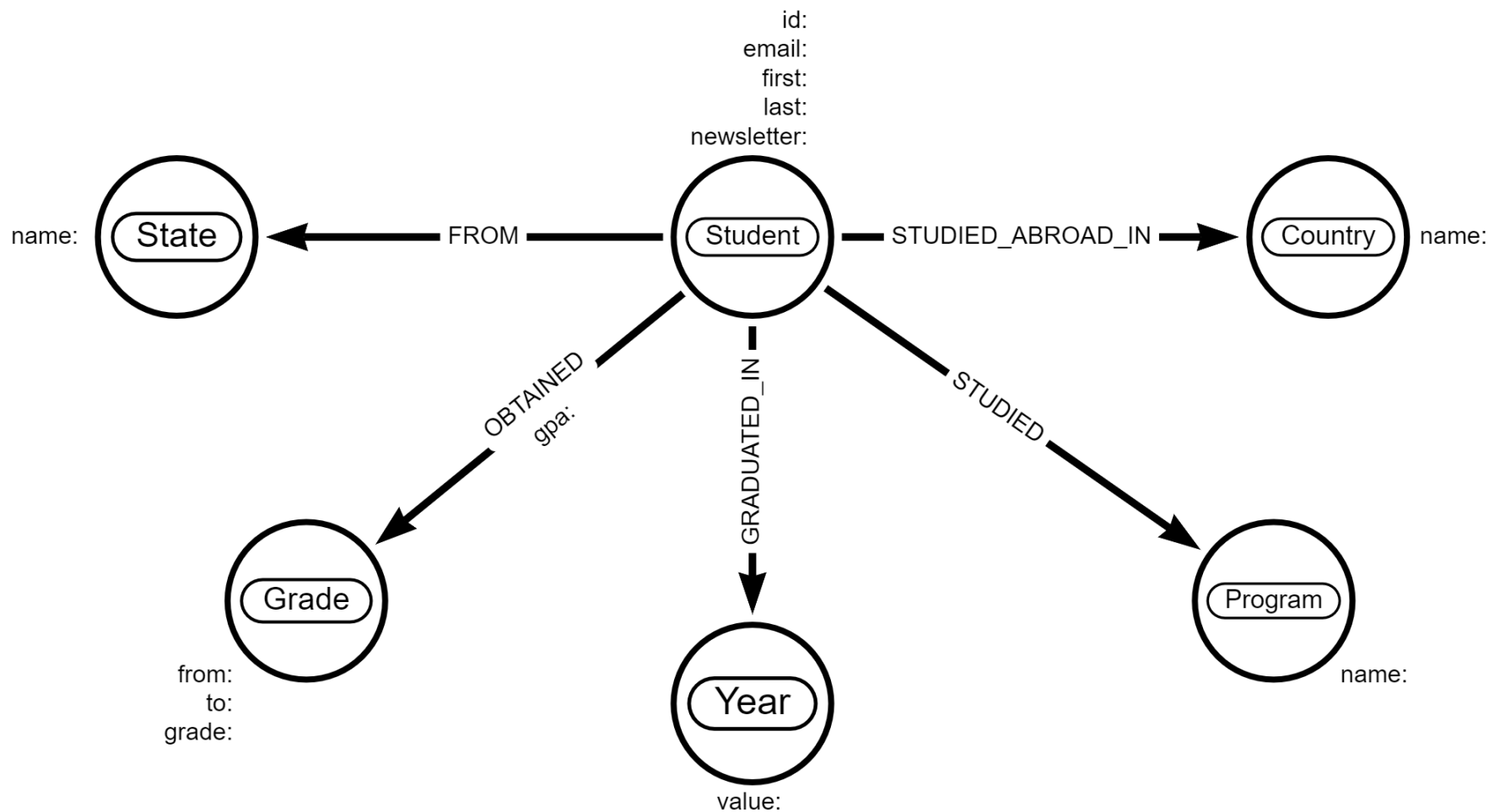
```
MATCH (n:Person {name: 'Alice'})
REMOVE n.age
```

Знайомство з Neo4

на прикладі

Різні операції з БД на прикладі

The Roux Academy data model



База даних для навчання

//Delete data (if any)

```
MATCH (n) DETACH DELETE n;
```

//Add constraints

```
CREATE CONSTRAINT IF NOT EXISTS FOR (student:Student) REQUIRE student.id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (state:State) REQUIRE state.name IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (gradYear:Year) REQUIRE gradYear.value IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (program:Program) REQUIRE program.name IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (country:Country) REQUIRE country.name IS UNIQUE;
```

//Load data. We will use several statements to minimise eager queries and flat file downloads.

//Note that this may not be how you'd typically do this!

```
LOAD CSV WITH HEADERS FROM "https://raw.githubusercontent.com/LinkedInLearning/learning-
neo4j-2483130/main/roux.csv" AS row
CREATE (student:Student{id:row.ID, last:row.LAST, first:row.FIRST, email:row.EMAIL,
newsletter:row.NEWSLETTER, state:row.STATE, grad:tointeger(row.GRADYEAR), prog:row.PROGRAM,
country:row.STUDIEDIN, gpa:toFloat(row.GPA)});
```

//Create and link the State node

```
MATCH (student:Student)
MERGE (state:State {name:student.state})
WITH student, state
CREATE (student)-[:FROM]->(state)
REMOVE student.state;
```

//Create and link the Year node

```
MATCH (student:Student)
MERGE (grad:Year {value:student.grad})
WITH student, grad
CREATE (student)-[:GRADUATED_IN]->(grad)
REMOVE student.grad;
```

//Create and link the Program node

```
MATCH (student:Student)
MERGE (prog:Program {name:student.prog})
WITH student, prog
CREATE (student)-[:STUDIED]->(prog)
REMOVE student.prog;
```

//Create and link the Country node. Skip the n/a values

```
MATCH (student:Student)
WHERE student.country <> 'n/a'
MERGE (country:Country {name:student.country})
WITH student, country
CREATE (student)-[:STUDIED_ABROAD_IN]->(country)
REMOVE student.country;
```

```
MATCH (student:Student)
WHERE student.country IS NOT NULL
REMOVE student.country;
```

//Create grade categories (Grade node) and link with GPA score as relationship property

```
WITH [[2.00,2.33,'C'],[2.33,2.67,'C+'],[2.67,3.00,'B'],[3.00,3.33,'B+'],[3.33,3.67,'A-'],
[3.67,4.00,'A'],[4.00,4.00,'A+']] AS ranges
UNWIND ranges AS range
CREATE (:Grade {from:range[0], to:range[1], grade:range[2]});
```

```
MATCH (student:Student), (grade:Grade)
WHERE student.gpa>=grade.from AND student.gpa < grade.to
CREATE (student)-[:OBTAINED {gpa:student.gpa}]->(grade)
REMOVE student.gpa;
```

MATCH – Співставлення вузлів

//Повернути всі вузли в базі даних (обмежити до 10 результатів)

MATCH (n) **RETURN** n **LIMIT** 10

//Повернути всі вузли, які мають мітку Student (обмежити до 10 результатів)

MATCH (n:Student) **RETURN** n **LIMIT** 10

//Повернути всі вузли, які мають властивість first (ім'я) зі значенням 'Alia'

MATCH (n {first: 'Alia'}) **RETURN** n

//Повернути всі вузли, які мають мітку Student і властивість first зі значенням 'Alia'

//Цей запит повинен бути швидшим, коли ми використовуємо мітку!

MATCH (n:Student {first: 'Alia'}) **RETURN** n

//Повернути всі вузли, які мають мітку Student і властивість first зі значенням 'Alia'

//Ми також можемо використовувати WHERE

MATCH (n:Student) **WHERE** n.first = 'Alia' **RETURN** n

//Повернути всі вузли Year після 2006 року

MATCH (n:Year) **WHERE** n.value > 2006 **RETURN** n

MATCH – Співставлення вузлів та відношень

//Повернути всіх студентів з ім'ям Alia, які навчалися за кордоном

```
MATCH (s:Student {first: 'Alia'})--(c:Country) RETURN *
```

//Повернути всіх студентів з ім'ям Alia, які навчалися за кордоном

//Цей запит повинен бути швидшим, коли ми використовуємо тип і напрямок відношення!

```
MATCH (s:Student {first: 'Alia'})-[:STUDIED_ABROAD_IN]->(c:Country) RETURN *
```

//Повернути всіх студентів з GPA вище 3.98

```
MATCH (s:Student)-[r:OBTAINED]->() WHERE r.gpa > 3.98 RETURN s.first, s.last, r.gpa
```

//Повернути всіх студентів з ім'ям Alia з GPA вище 3.0

```
MATCH (s:Student)-[r:OBTAINED]->() WHERE r.gpa > 3.0 AND s.first = 'Alia' RETURN s.first, s.last, r.gpa
```

//Повернути всіх студентів з ім'ям Alia з GPA вище 3.0

//Ми можемо змінити наші посилання, використовуючи AS

```
MATCH (s:Student)-[r:OBTAINED]->() WHERE r.gpa > 3.0 AND s.first = 'Alia' RETURN s.first AS Name, s.last AS `Family Name`, r.gpa AS GPA
```

Підрахунок та агрегація даних

Основні функції

count: Підраховує кількість вузлів або відношень у заданій ситуації.

collect: Збирає вузли, відношення, властивості або інші дані в масив для подальшого виведення або обробки.

size: Визначає розмір масиву або довжину рядка.

LIMIT: Обмежує кількість даних, які виводяться у запиті або використовуються для подальших запитів.

ORDER BY: Впорядковує дані за зростанням (за замовчуванням) або за спаданням.

Підрахунок та агрегація даних

//Скільки студентів навчалися за кордоном?

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(c:Country) RETURN count(s) AS `Students Abroad`
```

//Знайти та зібрати всіх студентів з ім'ям Alia або Colleen, які навчалися за кордоном, і в якій країні

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(c:Country) WHERE s.first = 'Alia' OR s.first = 'Colleen' RETURN collect(s.first + ' ' + s.last) AS Name, c.name AS Country
```

//Знайти кількість студентів, які навчалися за кордоном, по країнах

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(c:Country) WITH collect(s) AS students, c RETURN size(students) AS `Number of Students`, c.name AS Country
```

//Повернути 10 студентів у висхідному алфавітному порядку за ім'ям

```
MATCH (s:Student) RETURN s.first, s.last ORDER BY s.first LIMIT 10
```

//Знайти кількість студентів, які навчалися за кордоном, по країнах, впорядкованих від найбільшої до найменшої кількості

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(c:Country) WITH collect(s) AS students, c RETURN size(students) AS Students, c.name AS Country ORDER BY Students DESC
```

//Приклад з WITH та WHERE //Знайти країни, де навчалася щонайменше 150 студентів

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(c:Country) WITH collect(s) AS students, c WHERE size(students) >= 150 RETURN size(students) AS `Number of Students`, c.name AS Country
```

За межами одного стрибка

//Знайти всіх студентів, які навчалися за кордоном у Греції у 2011 році

```
MATCH (s:Student)-[:STUDIED_ABROAD_IN]->(:Country {name: "Greece"}), (s)-[:GRADUATED_IN]->(:Year {value: 2011})  
RETURN s.first, s.last
```

//Знайти всіх студентів, які навчалися за кордоном у Греції у 2011 році

```
MATCH (:Year {value: 2011})<-[:GRADUATED_IN]-(s:Student)-[:STUDIED_ABROAD_IN]->(:Country {name: "Greece"}) RETURN  
s.first, s.last
```

//Знайти всіх студентів, які знаходяться на відстані 2 переходів від студента Арделли Ітон

```
MATCH (s1:Student {first: "Ardella", last: "Eaton"})-[*0..2]-(s2:Student) RETURN DISTINCT s2.first, s2.last
```

//Знайти всіх студентів, які знаходяться на відстані 2 переходів від студента Арделли Ітон на основі програми, на якій вони навчаються

```
MATCH (s1:Student {first: "Ardella", last: "Eaton"})-[:STUDIED *0..2]-(s2:Student) RETURN s2.first, s2.last
```

//Знайти найкоротший шлях між Арделлою Ітон та Крістіаною Бун

```
MATCH (s1:Student {first: 'Ardella', last: 'Eaton'}), (s2:Student {first: 'Christiana', last: 'Boone'}), p = shortestpath((s1)-[*]-(s2))
```

//p - це шлях, що складається з вузлів та відношень

```
RETURN p
```

CREATE – Створення

//Різні типи вузлів

//CREATE () //Це створить порожній вузол без мітки та властивостей!

//CREATE (:Learner) //Це створить вузол з міткою Learner, але без властивостей

//Створення вузла

CREATE (:Learner {name: 'Lju'}) //Це створить вузол з міткою Learner та властивістю name=Lju

//Створення іншого вузла

CREATE (:Learner {name: 'Alia'})

//Створення відношення між існуючими вузлами

MATCH (lju:Learner {name: 'Lju'})

MATCH (alia:Learner {name: 'Alia'}) CREATE (lju)-[:KNOWS {since: 20210101}]->(alia)

//Створення двох вузлів та відношення за один раз!

CREATE (:Learner {name: 'Jo'})-[:KNOWS]->(:Learner {name: 'Su'})

//Повернути всіх учнів, щоб побачити, що ми створили

MATCH (l:Learner) RETURN l

Update & REMOVE – Оновлення та видалення

//Оновити дату, коли Lju познайомився з Alia

```
MATCH (:Learner {name: 'Lju'})-[r:KNOWS]->(:Learner {name: 'Alia'}) SET r.since = 19990101
```

//Додати властивість twitter handle для Lju

```
MATCH (lju:Learner {name: 'Lju'}) SET lju.twitter = 'ellazal'
```

//Додати мітку OffCampus для Alia

```
MATCH (alia:Learner {name: 'Alia'}) SET alia:OffCampus
```

//Повернути Alia та Lju, щоб побачити, що ми змінили

```
MATCH (l:Learner) WHERE l.name = 'Alia' OR l.name = 'Lju' RETURN l
```

//Видалити властивість twitter handle у Lju

```
MATCH (lju:Learner {name: 'Lju'}) REMOVE lju.twitter
```

//Видалити мітку OffCampus у Alia

```
MATCH (alia:Learner {name: 'Alia'}) REMOVE alia:OffCampus
```

//Повернути Alia та Lju, щоб побачити, що ми видалили

```
MATCH (l:Learner) WHERE l.name = 'Alia' OR l.name = 'Lju' RETURN l
```

DELETE – Видалення

```
CREATE (:Learner {name: 'Lju'})-[:KNOWS {since: 20210101}]->(alia:Learner {name: 'Alia'})
```

```
CREATE (:Learner {name: 'Jo'})-[:KNOWS]->(:Learner {name: 'Su'})
```

```
//DELETE все           //MATCH (n)           //DETACH DELETE n
```

```
//ВИДАЛИТИ тільки вузол Lju
```

```
MATCH (l:Learner {name: 'Lju'}) DETACH DELETE l
```

```
//ВИДАЛИТИ відношення між Jo та Su
```

```
MATCH (:Learner {name: 'Jo'})-[:KNOWS]->(:Learner {name: 'Su'}) DELETE r
```

```
//ВИДАЛИТИ всі вузли Learner
```

```
MATCH (l:Learner) DETACH DELETE l
```

```
//Створити вузли Jo та Su і відношення між ними
```

```
CREATE (:Learner {name: 'Jo'})-[:KNOWS]->(:Learner {name: 'Su'})
```

```
//Оновити відношення KNOWS до відношення FRIEND
```

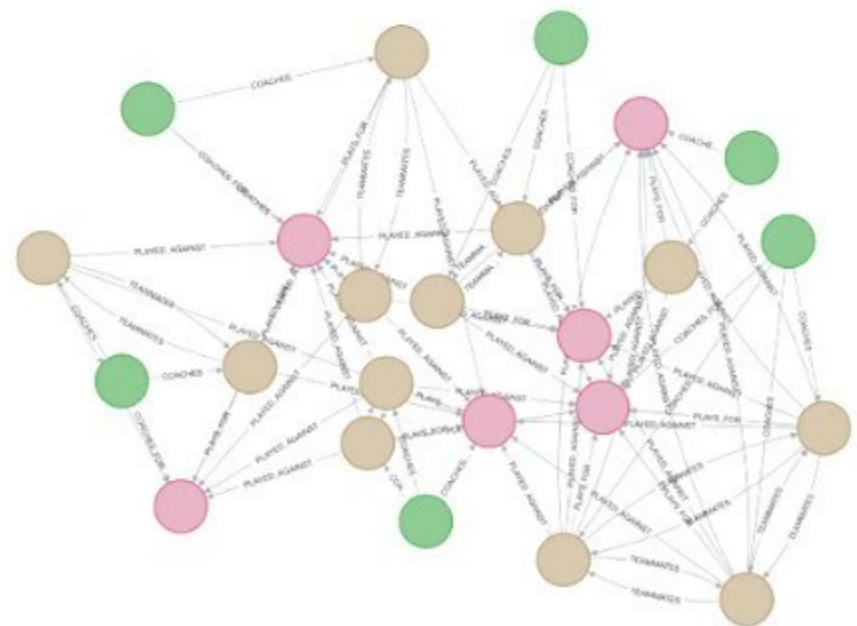
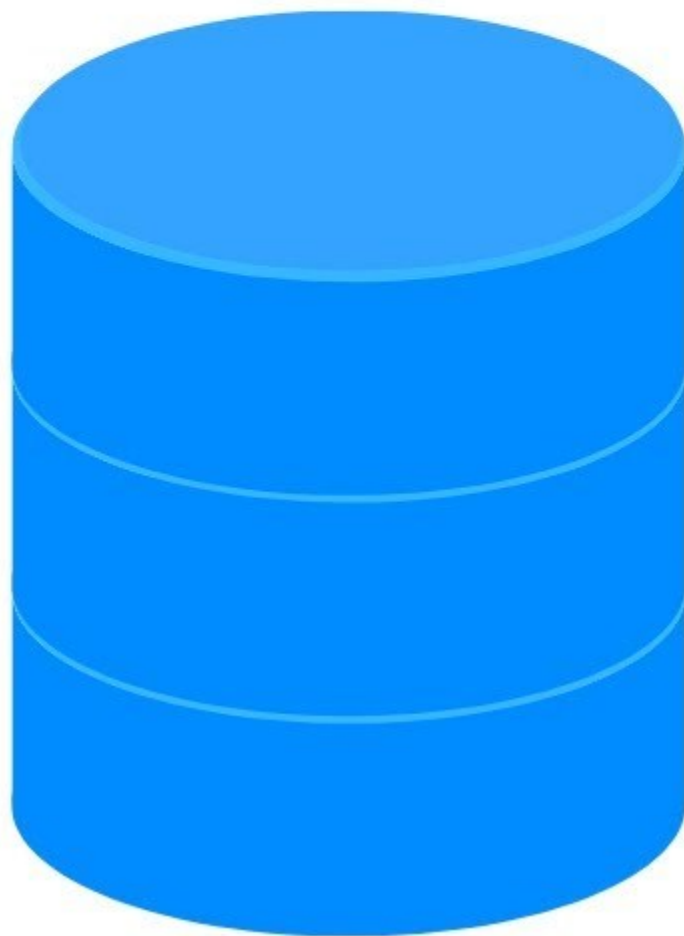
```
MATCH (l1:Learner {name: 'Jo'})-[:KNOWS]->(l2:Learner {name: 'Su'})
```

```
CREATE (l1)-[:FRIEND]->(l2) //створити нове відношення
```

```
DELETE r //видалити старе
```

```
//Переглянути новий тип відношення
```

```
MATCH (l1:Learner {name: 'Jo'})-[:r]->(l2:Learner {name: 'Su'}) RETURN *
```



ДЯКУЮ ЗА УВАГУ!